

# Scalable Frequent Itemset Mining for Online Retail: A Comparative Study and a Quantity-Aware Extension

Riccardo Fidanza

August 2026

## 1 Introduction

Online retail platforms generate large amounts of transactional data that can be analyzed to discover purchasing patterns through **Market Basket Analysis** and **Frequent Itemset Mining**. These techniques are widely used for applications such as product recommendation, cross-selling, and inventory management.

As the size of transactional datasets increases, the **scalability** of frequent itemset mining becomes a major challenge. Different algorithms adopt different strategies, resulting in significant differences in execution time and resource requirements.

Frequent itemset mining algorithms usually represent transaction as binary sets, considering only whether an item was purchased and ignoring the quantity purchased [4, 5]. This may hide relevant differences between ordinary consumer purchases and bulk acquisitions.

This project investigates three frequent itemset mining algorithms **A-Priori**, **SON**, and **FP-Growth** using the Online Retail dataset. Their performance is evaluated in terms of execution time by varying the number of available CPUs across different experiments, with the aim of understanding how their execution time changes as additional computational resources are provided. The A-Priori algorithm is implemented in a distributed form using Spark, while SON and FP-Growth are considered as alternative approaches to distributed frequent itemset mining. The analysis is then extended by introducing a quantity-aware version of A-Priori and SON, **which considers purchasing volume in addition to transaction frequency**.

## 2 Problem Description

### 2.1 Problem Statement

The problem addressed in this project concerns the efficient mining of frequent patterns from transactional retail data. As the number of transactions and distinct products increases, the computational cost of Frequent Itemset Mining can become significant, particularly when the minimum support threshold is lowered and the number of candidate itemsets grows rapidly.

A second challenge arises from the representation of transactions in classical Frequent Itemset Mining. Transactions are typically modeled as sets of items, capturing only whether a product was purchased while disregarding the quantity purchased. Consequently, transactions with substantially different purchase volumes may be treated identically, potentially hiding relevant purchasing patterns.

### 2.2 Dataset

The dataset used in this project is the *Online Retail* dataset [2], which contains transactional data from a UK-based online gift retailer collected between December 2010 and December 2011. The dataset consists of 541,909 records describing customer transactions and the products included in each purchase.

The main attributes relevant to the analysis are reported in Table 1. In particular, **InvoiceNo** identifies a transaction, while **StockCode** identifies the purchased product. The **Quantity** attribute specifies the number of units purchased, making it particularly relevant for the quantity-aware analysis. Additional attributes include product description, transaction date, unit price, customer identifier, and country of residence.

| Attribute   | Description                                                |
|-------------|------------------------------------------------------------|
| InvoiceNo   | 6-digit transaction ID ('C' prefix indicates cancellation) |
| StockCode   | 5-digit unique product ID                                  |
| Description | Product name                                               |
| Quantity    | Number of items purchased per transaction                  |
| InvoiceDate | Date and time of the transaction                           |
| UnitPrice   | Product price per unit (in sterling)                       |
| CustomerID  | 5-digit unique customer ID                                 |
| Country     | Customer's country of residence                            |

Table 1: Overview of the key dataset attributes.

The dataset is well suited to the considered problem because it contains both transaction-level product information and purchase quantities. It therefore supports the evaluation of classical Frequent Itemset Mining approaches as well as the proposed quantity-aware extensions.

## 3 Solution Overview

### 3.1 Project Workflow

The project consists of two analytical branches: an evaluation of the scalability and computational performance of A-Priori, SON, and FP-Growth on a 300x expanded dataset, along with the introduction and evaluation of quantity-aware extensions to the A-Priori and SON algorithms.

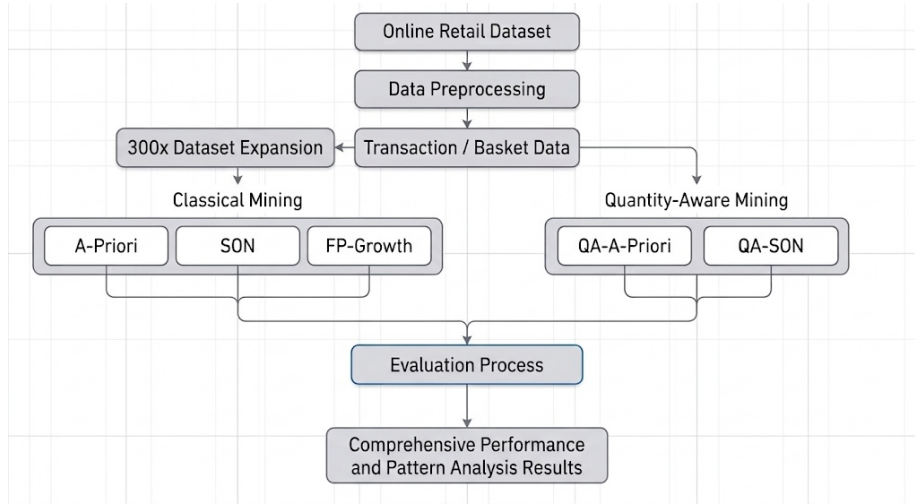


Figure 1: Project Workflow

### 3.2 Classical Mining Approaches

Three classical Frequent Itemset Mining algorithms are considered for the scalability analysis: **A-Priori**, **SON**, and **FP-Growth**. The three approaches rely on different strategies for discovering frequent itemsets and, therefore, provide complementary cases for evaluation in a distributed environment.

**A-Priori** [1] follows a level-wise candidate-generation strategy. At each iteration, candidate itemsets of a given size are generated from the frequent itemsets identified at the previous level and then counted and pruned according to the minimum support threshold. In this project, the classical algorithm is adapted to a distributed setting using a MapReduce-based implementation in PySpark, allowing candidate generation and frequency counting to be performed across distributed data partitions.

**SON** [6] also relies on candidate generation, but adopts a different distributed strategy. Instead of repeatedly mining the entire dataset, SON first identifies frequent itemsets independently within individual partitions and combines them into a global candidate set. A second pass over the complete dataset is then used

to verify the global frequency of these candidates. Thus, A-Priori is based on iterative global candidate generation and pruning, whereas SON exploits local mining followed by global verification.

**FP-Growth** [3] adopts a different pattern-growth strategy and avoids explicit candidate generation. Transactions are compressed into an FP-tree, from which frequent patterns are recursively extracted. This provides a fundamentally different approach to frequent itemset mining and serves as a comparison with the candidate-generation methods.

### 3.3 Quantity-Aware Extension

The second part of the project extends the frequent itemset mining process by incorporating purchase quantities into the analysis. The frequency of each itemset remains the criterion for determining whether a pattern is frequent, while additional metrics are computed to characterize the associated purchasing volume.

The extension is applied to both A-Priori and SON, preserving the underlying mining strategies while enriching the output with quantity-based information. This enables the identified patterns to be analyzed according to both their transaction frequency and the volume associated with the corresponding purchases.

## 4 Implementation

### 4.1 Data Preprocessing

Before applying the Frequent Itemset Mining algorithms, the raw dataset is cleaned and transformed into a transactional representation. Cancellation records, identified by invoice numbers starting with "C", are removed, together with records having non-positive quantities or missing product descriptions. Records with missing `CustomerID` values are retained, as customer identification is not required for transaction-level Market Basket Analysis. The remaining records are grouped by `InvoiceNo`, with the corresponding `StockCode` values used to construct the product set associated with each transaction. Transactions containing only a single product are excluded, as they cannot contribute to multi-item frequent itemsets. The resulting collection of baskets is then cached in memory to improve the performance of the subsequent iterative mining algorithms. This preprocessing step produces the transactional representation used by all the considered mining approaches, while preserving the original `Quantity` information required by the quantity-aware extensions.

### 4.2 A-Priori

A-Priori is a level-wise Frequent Itemset Mining algorithm based on the **downward closure property**, which states that every subset of a frequent itemset

must also be frequent. Starting from frequent 1-itemsets, the algorithm iteratively generates candidate  $k$ -itemsets from the frequent  $(k - 1)$ -itemsets, counts their occurrences, and removes candidates whose support is below the minimum support threshold. The process continues until no new frequent itemsets can be generated.

For this project, A-Priori is adapted to a distributed setting using PySpark. Candidate generation is performed on the driver node, where valid candidates are constructed from the frequent itemsets identified in the previous iteration. The resulting candidate set is then broadcast to the worker nodes, where the support of each candidate is counted over the distributed transaction dataset.

As illustrated in Figure 2, each iteration of the distributed A-Priori algorithm consists of the following steps:

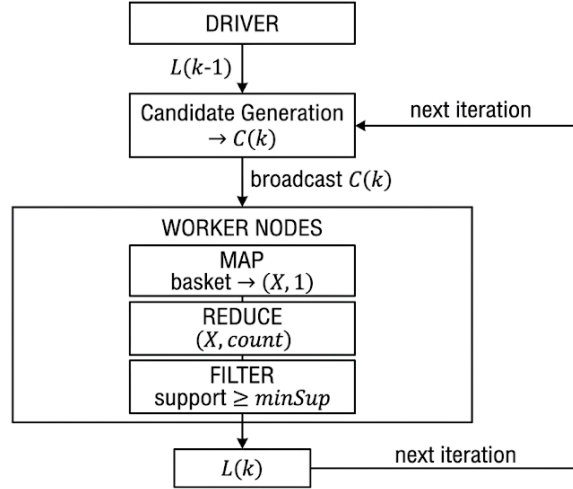


Figure 2: Distributed implementation of A-Priori using a MapReduce-based support-counting phase.

1. **Candidate Generation:** The frequent itemsets  $L_{k-1}$  obtained in the previous iteration are available on the driver. These itemsets are used to generate the candidate set  $C_k$ .
2. **Candidate Broadcast:** The generated candidates  $C_k$  are broadcast from the driver to all worker nodes, so that each worker can use the same candidate set to process its local baskets.
3. **Map:** Each worker examines its assigned baskets and checks which candidates are contained in each basket. For each candidate itemset  $X$  contained in a basket, the pair  $(X, 1)$  is emitted.
4. **Reduce:** The pairs generated by the workers are grouped by candidate itemset and their values are summed. This produces the global support

count for each candidate.

5. **Filtering:** Candidates whose support is below the minimum support threshold are discarded. The remaining candidates form the frequent itemsets  $L_k$ .
6. **Next Iteration:** The frequent itemsets  $L_k$  are returned to the driver and used to generate the candidates  $C_{k+1}$ . The process is repeated until no new frequent itemsets are found.

### 4.3 SON

SON is a distributed Frequent Itemset Mining algorithm designed to identify globally frequent itemsets through two MapReduce phases. Its key property is that an itemset that is frequent in the complete dataset must be frequent in at least one of its partitions.

**Phase 1 – Local Candidate Generation.** The dataset is divided into partitions, each processed independently. A local A-Priori procedure is applied using MapReduce: the Map stage counts itemsets within each partition, while the Reduce stage aggregates these counts to identify locally frequent itemsets. Their union forms the global candidate set.

**Phase 2 – Global Counting.** The global candidate set is evaluated over the complete dataset. The Map stage counts the occurrences of each candidate in each partition, while the Reduce stage aggregates these partial counts to obtain their global support. Candidates satisfying the global minimum support are retained.

The two-phase structure allows candidate generation to be performed locally while requiring only a final global counting step to verify the frequency of the resulting candidates.

### 4.4 FP-Growth

FP-Growth is a Frequent Itemset Mining algorithm that avoids the explicit candidate generation used by A-Priori and SON. Instead, transactions are compressed into a compact *FP-tree* structure, which preserves the frequency and co-occurrence information of the items. Frequent itemsets are then extracted by recursively mining the tree and constructing conditional pattern bases.

In this project, the distributed implementation provided by PySpark is used through the `pyspark.ml.fpm` library. The transaction baskets obtained during preprocessing are provided as input to the FP-Growth model, together with the selected minimum support threshold. The resulting model directly produces the frequent itemsets and their support counts.

Unlike the custom A-Priori and SON implementations, no candidate generation or explicit MapReduce support-counting procedure is implemented for FP-

Growth. The distributed execution and pattern-growth operations are handled by the PySpark implementation.

#### 4.5 Quantity-Aware A-Priori and SON

The quantity-aware extension augments the classical mining process by associating each itemset occurrence with the quantity purchased in the corresponding transaction. **The frequency-based support is kept unchanged and is still used for pruning, so that the original mining logic is preserved.**

For an itemset  $X$  contained in a transaction  $t$ , the effective co-purchased volume is defined as

$$V(X, t) = \min_{i \in X} q_i.$$

The minimum quantity is used because the joint purchase volume of an itemset is limited by the item purchased in the smallest quantity. Thus,  $V(X, t)$  represents the maximum number of complete instances of  $X$  that can be formed within transaction  $t$ .

Since purchase quantities can vary over a wide range, a logarithmic transformation is applied:

$$L(X, t) = \log(1 + V(X, t)).$$

The transformation reduces the influence of very large quantities while preserving their relative ordering.

The resulting quantity information is aggregated over all transactions containing  $X$ . Three additional metrics are computed. The total log volume is defined as

$$TL(X) = \sum_{t \in D_X} L(X, t),$$

and measures the overall purchasing intensity associated with the itemset across all its occurrences.

The maximum co-purchased volume is defined as

$$M(X) = \max_{t \in D_X} V(X, t),$$

and identifies the largest quantity of complete instances of  $X$  purchased in a single transaction.

Finally, the average log volume is defined as

$$AvgL(X) = \frac{TL(X)}{S(X)},$$

which measures the average purchasing intensity per transaction containing  $X$  and allows patterns with different frequencies to be compared more fairly.

The quantity-aware extension modifies the MapReduce counting phase while leaving the candidate generation and pruning logic unchanged. In the classical implementation, each occurrence of a candidate itemset produces:

$$\text{Map: } (X, 1) \quad \text{Reduce: } (X, S(X)).$$

In the quantity-aware version, the same occurrence produces additional quantity information:

$$\text{Map: } (X, (1, L(X, t), V(X, t))),$$

which is then aggregated during the Reduce phase as

$$\text{Reduce: } (X, (S(X), TL(X), M(X))).$$

For A-Priori, this modification is applied at each candidate-counting iteration. For SON, the local candidate-generation phase remains unchanged, and the extended counting is applied during the global verification phase. In both cases, the final pruning step remains based only on  $S(X) \geq \text{minSup}$ .

To illustrate the additional information provided by these metrics, consider two itemsets  $X$  and  $Y$  that both occur in 100 transactions:

$$S(X) = S(Y) = 100.$$

Classical Frequent Itemset Mining therefore considers the two patterns equally frequent. Suppose, however, that  $X$  is typically purchased in small quantities, while  $Y$  is often purchased in bulk. In this case,  $Y$  will have higher  $TL(Y)$  and  $AvgL(Y)$  values, indicating greater overall and average purchasing intensity. Furthermore, if one transaction contains a particularly large purchase of  $Y$ , this will be reflected by a high  $M(Y)$ . **The three metrics therefore provide information that cannot be obtained from support alone, distinguishing patterns that are equally frequent but associated with substantially different purchasing behavior.**

## 5 Experimental Evaluation

### 5.1 Dataset Analysis

The initial analysis of the raw dataset identified several data quality issues (Table 2) that required preprocessing before the mining phase. In particular, 135,080 records have a missing `CustomerID`, 9,288 correspond to cancellations, 10,624 contain non-positive quantities, and 1,454 have a missing product description.

Cancellation records, non-positive quantities, and records with missing descriptions were removed, while transactions with missing `CustomerID` were retained

because customer identification is not required for transaction-level Market Basket Analysis. After preprocessing, 530,693 valid records remained and were aggregated into 18,338 multi-item baskets.

The resulting baskets constitute the input used by all the considered mining algorithms. The original **Quantity** information is also preserved for the subsequent quantity-aware analysis.

| Anomaly Type                                          | Number of Records |
|-------------------------------------------------------|-------------------|
| Guest Users (Missing CustomerID)                      | 135,080           |
| Returns and Cancellations (InvoiceNo starts with 'C') | 9,288             |
| Negative or Zero Quantity                             | 10,624            |
| Missing Descriptions / Warehouse Errors               | 1,454             |

Table 2: Data profiling of anomalies in the raw dataset.

## 5.2 Experimental Setup

### 5.2.1 Computational Environment

The experiments were conducted on a remote university machine equipped with an Intel Core i9-10900X CPU, providing 10 physical cores and 20 logical CPUs.

### 5.2.2 Dataset Scaling

To evaluate the behavior of the algorithms on a substantially larger workload, the preprocessed transactional dataset was replicated 300 times. This produces a synthetic dataset that preserves the structure and characteristics of the original transactions while increasing the overall data volume. The replicated dataset is used exclusively for the scalability experiments.

### 5.2.3 Scalability Configuration

The scalability analysis is performed by varying the available computational resources while keeping the dataset and the number of Spark partitions fixed. Three configurations are considered: 20, 10, and 4 execution threads, with 40 partitions in all cases. This setup allows us to isolate the effect of increasing computational resources on the execution time and scalability of the algorithms. The algorithms are evaluated at minimum support thresholds of 3.0%, 2.5%, and 2.0%.

### 5.2.4 Evaluation Metrics

The evaluation considers both the overall performance of the algorithms and the internal distribution of their computational cost. The main metric is **total execution time**, which measures the time required to complete the mining process. The effect of different minimum support thresholds is also analyzed to evaluate how the computational cost changes as the search space increases.

Table 3: Hardware configurations used for the scalability experiments.

| Execution Threads | Spark Partitions | Dataset         |
|-------------------|------------------|-----------------|
| 20                | 40               | 300× replicated |
| 10                | 40               | 300× replicated |
| 4                 | 40               | 300× replicated |

The number of **frequent itemsets discovered** is used to verify the consistency of the different implementations. In addition, the internal execution time of each algorithm is analyzed: for A-Priori, the time spent at each mining level is considered; for FP-Growth, the time required to build the FP-tree is compared with the time required for pattern mining; for SON, the execution time of the local candidate-generation (phase 1) is compared with that of the global counting phase (phase 2).

Finally, the scalability analysis evaluates how execution time changes when the available computational resources are reduced from 20 to 10 and 4 execution threads while keeping the number of Spark partitions fixed at 40.

For the quantity-aware analysis, the additional metrics considered are the maximum co-purchased volume  $M(X)$  and the average log volume  $AvgL(X)$ , which are used to characterize the purchasing intensity of the discovered patterns.

### 5.3 Classical Mining Algorithms

#### 5.3.1 Performance at 20 Cores

The performance of A-Priori, SON, and FP-Growth is first analyzed using the full 20-core configuration. Dashboard 3 summarizes the execution time of the three algorithms at the three considered minimum support thresholds, together with a breakdown of their internal computational costs.

A-Priori requires 50.42 s, 150.64 s, and 486.55 s at 3.0%, 2.5%, and 2.0% minimum support, respectively. The level-wise breakdown shows that most of the execution time is concentrated at Level 2, requiring 43.24 s, 146.43 s, and 482.01 s at the three thresholds. Levels 1 and 3 contribute only a small fraction of the total time. This indicates that the main computational cost of the distributed A-Priori implementation is associated with the generation and support counting of candidate 2-itemsets, whose number increases substantially as the support threshold decreases.

SON shows a similar trend, with total execution times of 52.61 s, 165.94 s, and 513.65 s. The phase breakdown reveals that almost all of the execution time is spent in Phase 1, while Phase 2 remains comparatively small. At 2.0% support, for example, Phase 1 requires 510.07 s compared with only 3.58 s for Phase 2. The main computational bottleneck of SON is therefore the local candidate-generation stage, especially at low support thresholds.

FP-Growth is considerably faster, requiring 14.61 s, 20.43 s, and 27.87 s at 3.0%, 2.5%, and 2.0% support. Its execution is divided between FP-tree construction and pattern mining, with the mining phase accounting for most of the total time. Unlike A-Priori and SON, the increase in execution time as the support threshold decreases is relatively limited. This behavior is consistent with the absence of explicit candidate generation in FP-Growth.

Overall, at 20 cores, FP-Growth provides the best performance across all support thresholds, while A-Priori and SON become increasingly expensive as the minimum support decreases. The internal breakdowns further show that the computational cost of the two candidate-generation approaches is concentrated in specific stages, particularly A-Priori Level 2 and SON Phase 1.

### 5.3.2 Scalability Analysis

The scalability of the three algorithms is evaluated by varying the available computational resources from 20 to 10 and 4 cores, while keeping the number of Spark partitions fixed at 40. Dashboard 4 reports the total execution time for the three minimum support thresholds.

A-Priori and SON show a clear increase in execution time as the available resources are reduced. The effect becomes more pronounced as the minimum support threshold decreases. In particular, at 2.0% support, reducing the execution threads from 20 to 4 causes the execution time of A-Priori and SON to increase substantially. This behavior is consistent with the higher computational cost of candidate generation and support counting at lower support thresholds, making these algorithms more dependent on the available parallelism.

The two candidate-generation approaches also show similar scalability trends, although SON consistently requires slightly more time than A-Priori. This difference becomes more evident at lower support thresholds, where the local candidate-generation phase of SON becomes increasingly expensive.

FP-Growth exhibits a markedly different behavior. Its execution time remains comparatively stable across the tested hardware configurations and is much less affected by the reduction in available threads. This indicates that, for the considered workload, FP-Growth is less dependent on increasing CPU parallelism than the candidate-generation approaches.

Overall, the results show that A-Priori and SON benefit more strongly from additional computational resources, particularly when the minimum support threshold is low. FP-Growth, while already considerably faster, shows a smaller variation across the tested configurations, indicating a lower sensitivity to the available parallelism.

### 5.3.3 Frequent Itemsets Discovered

Table 4 summarizes the exact number of frequent itemsets discovered by each algorithm at the evaluated support thresholds.



Figure 3: Performance analysis of A-Priori, SON, and FP-Growth using 20 execution threads.

| Minimum Support | A-Priori / SON | FP-Growth |
|-----------------|----------------|-----------|
| 3.0%            | 166            | 166       |
| 2.5%            | 276            | 276       |
| 2.0%            | 469            | 469       |

Table 4: Number of frequent itemsets discovered at varying support thresholds.

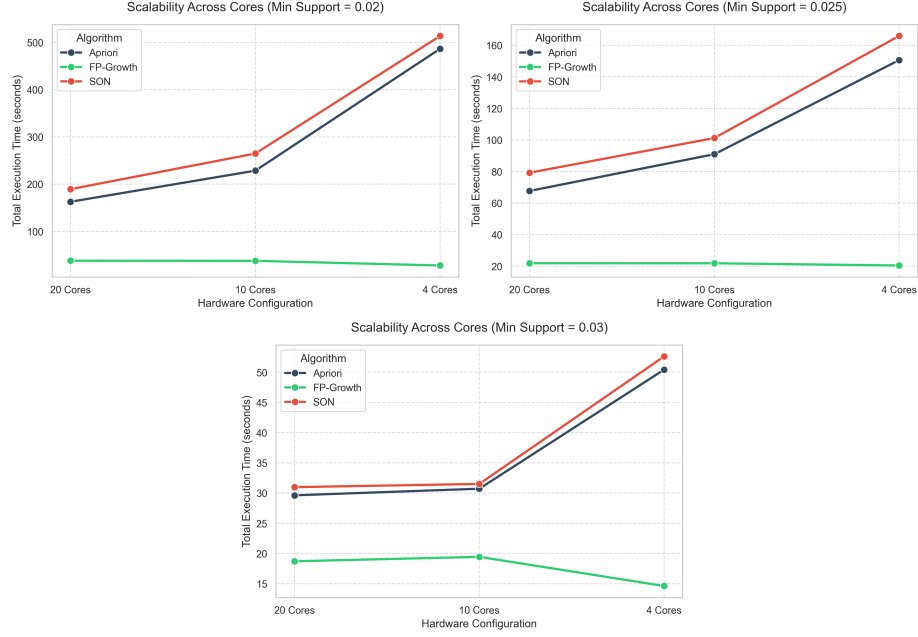


Figure 4: Scalability analysis

As demonstrated in Table 4, all three algorithms yielded an identical number of frequent itemsets across the evaluated minimum support thresholds. This exact correspondence confirms the algorithmic correctness and consistency of the respective implementations.

#### 5.4 Quantity-Aware A-Priori and SON

Table 5 reports the ten itemsets with the highest support among the quantity-aware results. In addition to support, the table includes the average log volume ( $Avg\_Log\_Qty$ ), the maximum co-purchased quantity ( $Max\_Qty$ ), and the item-set length.

| Itemset                                                                    | Support | Avg Log Qty | Max Qty | Length |
|----------------------------------------------------------------------------|---------|-------------|---------|--------|
| JUMBO BAG PINK<br>POLKADOT + JUMBO BAG<br>RED RETROSPOT                    | 825     | 1.95        | 800     | 2      |
| GREEN REGENCY TEACUP<br>AND SAUCER + ROSES<br>REGENCY TEACUP AND<br>SAUCER | 768     | 1.70        | 192     | 2      |
| JUMBO STORAGE BAG SUKI<br>+ JUMBO BAG RED<br>RETROSPOT                     | 724     | 1.82        | 200     | 2      |
| JUMBO SHOPPER VINTAGE<br>RED PAISLEY + JUMBO<br>BAG RED RETROSPOT          | 680     | 1.78        | 200     | 2      |
| LUNCH BAG SUKI DESIGN +<br>LUNCH BAG RED<br>RETROSPOT                      | 655     | 1.82        | 150     | 2      |
| LUNCH BAG BLACK SKULL<br>+ LUNCH BAG RED<br>RETROSPOT                      | 641     | 1.83        | 100     | 2      |
| ALARM CLOCK BAKELIKE<br>GREEN + ALARM CLOCK<br>BAKELIKE RED                | 640     | 1.61        | 300     | 2      |
| PINK REGENCY TEACUP<br>AND SAUCER + GREEN<br>REGENCY TEACUP AND<br>SAUCER  | 633     | 1.70        | 192     | 2      |
| LUNCH BAG RED<br>RETROSPOT + LUNCH BAG<br>PINK POLKADOT                    | 606     | 1.85        | 100     | 2      |
| PINK REGENCY TEACUP<br>AND SAUCER + ROSES<br>REGENCY TEACUP AND<br>SAUCER  | 599     | 1.72        | 192     | 2      |

Table 5: Top 10 frequent itemsets according to support with quantity-aware metrics. Highlighted rows correspond to the examples discussed in the text.

The quantity-aware metrics provide a more detailed view of the discovered purchasing patterns by distinguishing frequency from purchasing volume. The itemset *JUMBO BAG PINK POLKADOT + JUMBO BAG RED RETROSPOT* is the most frequent pattern in the table, with a support of 825, and it also has the highest average log quantity, equal to 1.95. Its maximum co-purchased quantity reaches 800, **indicating that this pair is not only frequently purchased, but can also be associated with particularly large purchases.**

In contrast, *GREEN REGENCY TEACUP AND SAUCER + ROSES REGENCY TEACUP AND SAUCER* has the second-highest support, with 768 occurrences, but a lower average log quantity of 1.70 and a maximum quantity

of 192. The pattern is therefore highly frequent, while the typical purchasing volume is lower than that of the most frequent itemset. **This shows that a high support value does not necessarily imply a high purchasing quantity.**

The *ALARM CLOCK BAKELIKE GREEN + ALARM CLOCK BAKELIKE RED* pair provides a further example. Its support is lower, at 640, and its average log quantity is the lowest among the highlighted patterns, at 1.61. However, its maximum quantity reaches 300. **This suggests that the pair is not typically purchased in large quantities, but larger-volume transactions can still occur.**

The lunch-bag itemsets show a different characteristic. *LUNCH BAG SUKI DESIGN + LUNCH BAG RED RETROSPOT* has a support of 655 and an average log quantity of 1.82, while other lunch-bag pairs have very similar values. Their maximum quantities, however, range from 100 to 150. Thus, **patterns with similar frequency and average purchasing intensity can still differ in the volume reached by individual transactions.**

These comparisons show that the three metrics capture different dimensions of purchasing behavior. Support measures how frequently an itemset occurs,  $AvgL(X)$  indicates its typical purchasing intensity, and  $M(X)$  captures the largest observed co-purchased volume. As a result, **an itemset can be highly frequent but associated with relatively small quantities, or less frequent while still reaching high quantities in individual transactions.**

This additional information is relevant in a retail context because frequent purchasing and high purchasing volume do not necessarily identify the same products. Support can highlight products that are consistently bought together, while the quantity-aware metrics can reveal whether these associations are usually associated with small purchases or whether they occasionally correspond to substantially larger volumes.

The scatter plot in Dashboard 5 illustrates this relationship by combining support and average log quantity, with bubble size representing the maximum quantity. The histogram in Dashboard 5 further shows that maximum quantities are unevenly distributed, with most itemsets associated with moderate values and a smaller number reaching substantially higher quantities.

Overall, the quantity-aware extension shows that support alone provides only a partial description of purchasing behavior. Combining support with  $AvgL(X)$  and  $M(X)$  makes it possible to distinguish frequent patterns with different purchasing intensities and to identify associations that can occasionally involve substantially larger purchase volumes.

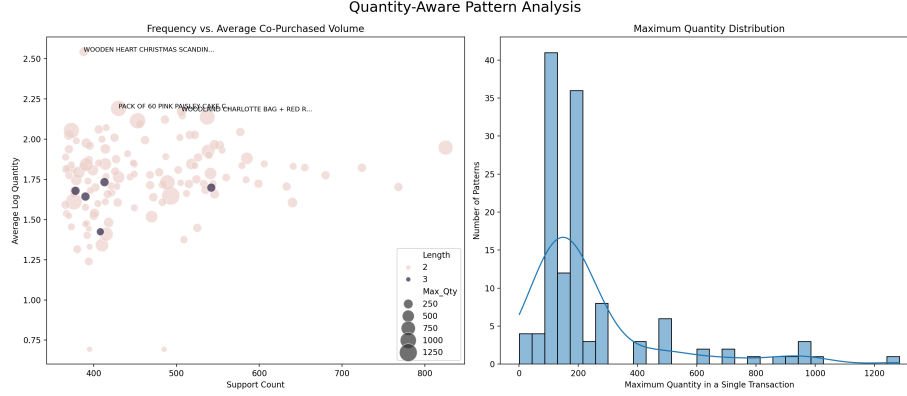


Figure 5: Quantity aware dashboard

## 6 Conclusion

This project investigated the scalability of Frequent Itemset Mining for online retail data and extended the analysis by incorporating purchasing quantities. Three approaches were considered: A-Priori, SON, and FP-Growth. The algorithms were evaluated in a distributed PySpark environment using different numbers of execution threads, with the objective of analyzing their computational behavior as the available resources changed.

The experimental results showed clear differences between the considered approaches. A-Priori and SON rely on candidate generation and support counting, while FP-Growth adopts a pattern-growth strategy that avoids explicit candidate generation. The results also showed that the computational cost of candidate-generation approaches increases substantially as the minimum support threshold is lowered, due to the larger number of candidates that must be generated, counted, and verified. FP-Growth exhibited a different behavior, with its execution relying on FP-tree construction and subsequent pattern mining. Despite these differences, all three implementations produced the same number of frequent itemsets at the evaluated support thresholds, confirming the consistency of the obtained results.

The scalability analysis further showed that reducing the available execution threads affects the execution time of the algorithms, although the magnitude of the effect differs across approaches. This highlights that the scalability of Frequent Itemset Mining depends not only on the amount of computational resources available, but also on the internal structure of the algorithm and on the amount of candidate and pattern-processing work generated by the selected support threshold.

The second part of the project addressed a limitation of the classical set-based representation, in which transactions indicate whether an item was purchased

but do not consider how many units were purchased. Quantity-aware extensions of A-Priori and SON were therefore introduced while preserving the original support-based definition of frequent itemsets. Additional metrics, namely average log quantity and maximum co-purchased quantity, were used to characterize the purchasing volume associated with the discovered patterns.

The quantity-aware results showed that support alone does not provide a complete description of purchasing behavior. For example, *JUMBO BAG PINK POLKADOT + JUMBO BAG RED RETROSPOT* achieved the highest support of 825, together with an average log quantity of 1.95 and a maximum quantity of 800. In contrast, *GREEN REGENCY TEACUP AND SAUCER + ROSES REGENCY TEACUP AND SAUCER* had a similarly high support of 768 but a lower average log quantity of 1.70 and a maximum quantity of 192. These differences demonstrate that itemsets with similar frequency can be associated with substantially different purchasing volumes.

Overall, the project demonstrates that classical Frequent Itemset Mining can be effectively used to identify recurring product associations, while the quantity-aware extension provides an additional perspective on the intensity of those purchases. The combination of frequency, average purchasing intensity, and maximum observed volume therefore offers a more informative characterization of retail purchasing patterns than support alone, while retaining the original mining logic and allowing the discovered associations to be interpreted from both frequency and quantity perspectives.

## References

- [1] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases*, pages 487–499, 1994.
- [2] Daqing Chen. Online Retail. UCI Machine Learning Repository, 2015. DOI: <https://doi.org/10.24432/C5BW33>.
- [3] Jiawei Han, Jian Pei, and Yiwen Yin. Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 1–12, 2000.
- [4] Tzung-Pei Hong, Chan-Sheng Kuo, and Sheng-Chai Chi. Mining association rules from quantitative data. *Intelligent Data Analysis*, 3(5):363–376, 1999.
- [5] Ping-Yu Hsu, Yen-Liang Chen, and Chun-Ching Ling. Algorithms for mining association rules in bag databases. *Information Sciences*, 166(1–4):31–47, 2004.
- [6] Ashok Savasere, Edward Omiecinski, and Shamkant B. Navathe. An efficient algorithm for mining association rules in large databases. In *Proceedings of the 21st International Conference on Very Large Data Bases*, pages 432–444, 1995.